BOCAL

Music practice, made legible.

Product strategy · competitor parity · 50 persona workflows
Native Android source · static web build · 35 interactive woodwind models

HANDOFF STATUS

Working static reference and structurally validated assets. Native source supplied. No fake APK; physical Android compilation and specialist fingering sign-off remain explicit gates.

REFERENCE HANDOFF / 10 AUGUST 2026

Contents

Companion files: 84-row TE parity CSV, detailed baseline delta, source code, ready static build, 35-model pack and native Android Studio project.

Version 0.2 reference handoff · 10 August 2026

Executive decision

Bocal should compete as **the fastest path from a musical intention to a useful next action**. TonalEnergy has a formidable breadth of documented features; copying its screen density would reproduce the problem Bocal is supposed to solve. The winning shape is a task-first shell with six stable verbs—**Tune, Lab, Pulse, Sound, Analyze, Practice**—and saved workspaces that reveal advanced controls only when the job requires them.

This package is a serious starting system, not a false "full parity complete" claim:

- A standalone static web app builds successfully and implements live tuning, transposition, reference pitch, pitch history, a metronome, tone generation, basic signal analysis, practice timing/journaling, local recording/download, and interactive 3D.
- An original asset generator produces **35 educational woodwind GLBs with 465 named interactive controls**. Ten are saxophones. All models pass structural validation.
- The alto saxophone is the first note-mapped instrument, with a core written B-flat3–F6 chart in the application data. The other models expose recognizable parts and controls but intentionally withhold note-level claims pending specialist validation.
- A native Kotlin/Jetpack Compose Android Studio project contains the code path I would use for the first APK: native microphone tuner, metronome, reference tone, analysis snapshot, practice data, and a local `WebViewAssetLoader` lab. It declares no Internet permission.
- No APK is included because the execution environment has Java but no Android SDK, Platform/Build Tools, `aapt2`, Gradle executable or wrapper JAR. The build and APK verification scripts are supplied. Renaming a ZIP to `.apk` would be dishonest and unusable.
- The hosted `chatgpt.site` URL still reflects the previous Cloudflare Worker failure. The inspected failure path initialized Three.js/GLTF loading code in a server/Worker context. The source copy includes a client-only dynamic-load fix, but this handoff does not claim the public deployment changed. The standalone build avoids that Worker runtime entirely.

1. Product thesis

1.1 The user promise

> Open Bocal, choose what you are trying to improve, and receive one legible correction—without creating an account or learning the app first.

Three layers make that defensible:

1. **Speed layer** — persistent instrument context, written/concert clarity, one primary action, saved workspaces and sensible presets.
2. **Teaching layer** — interactive instrument models, note-to-fingering and fingering-to-note, part/finger vocabulary, transposition explanations and validated alternatives.
3. **Evidence layer** — pitch trace, signal descriptors, metronome/timer, recordings and a local practice history that lead to an action rather than a decorative score.

TonalEnergy competes on breadth and depth. Bocal must reach comparable professional capability while reducing navigation cost and adding a learning representation the incumbent does not center: the instrument itself.

1.2 Product principles

- **The task is the navigation.** Six verbs are easier to recall than a feature taxonomy.
- **Written and sounding pitch are never ambiguous.** Instrument context is prominent and atomic.
- **Describe evidence; do not grade artistry.** “Upper harmonics increased” is evidence. “Your tone is bad” is not.
- **Progressive disclosure preserves professional depth.** Beginner mode changes defaults and density, not the underlying engine.
- **Offline is a trust and reliability feature.** Core practice must work in a rehearsal room, school or venue with no account/network.
- **Accessibility is an equivalent interaction, not a later label.** Color, 3D touch and audio each require another route to the same decision.
- **Educational content has release governance.** A polished wrong fingering is worse than a plain correct chart.

1.3 Primary success measures

Outcome	Measure	Initial gate
Faster comprehension	Median time from cold open to first stable written-pitch correction	Under 15 seconds for a returning user; benchmark against the incumbent on the same device
Accurate tuning	Gross pitch accuracy, octave error and cent error on a labeled woodwind corpus	Publish measured supported range; no unsupported C0–C8 claim
Reliable pulse	Tempo error and drift over 30 minutes; recovery from audio focus changes	No accumulated beat drift outside the defined threshold
Useful learning	Fingering recall after a short 3D lesson versus a static chart	Pre-register task; test note and reverse lookup separately
Lower navigation cost	Task completion time/errors for tune, drone, preset, record and review	Bocal faster on at least four of five core tasks without hiding capability
Trust	Percentage of users who correctly understand where audio/data goes	At least 90% after onboarding/privacy check
Quality	Crash-free sessions, audio-route failures and WebView model failures	Release threshold defined before beta, segmented by device/API

2. Research and incumbent definition

The primary competitive source is TonalEnergy's own Android/Desktop user guide, cross-checked with its mobile product page. That documentation establishes a much larger parity surface than “tuner + metronome”: target/bar tuning, tenths-of-cent display, volume/tone meters, notation/transposition/reference A, modes/ranges/damping, provided and custom temperaments, pitch tracking/timed events/activity; a deeply programmable metronome; tone exercises and sound libraries; waveform/spectral/harmonic/staff/interval analysis; recording organization/editing/import/export/time/pitch manipulation; external/MIDI/Link/BodyBeat integrations; personalization, languages and accessibility.

The full research ledger is in research/Research_Sources_and_Method.md. The feature-by-feature competitive contract is research/TE_Parity_Matrix.csv, with **84 rows** and an acceptance-evidence column. This matters because parity should be a test plan, not a marketing adjective.

2.1 What Bocal must match

Capability group	Parity expectation
Tuning	Precise chromatic detection across a declared range, transposition, adjustable A, temperament/key context, sensitivity/damping, pitch/session history and professional readability.
Pulse	Common and unusual meters/subdivisions, tap/numeric control, accents/sounds/visual/haptics, count-ins, silence exercises, reusable presets, sequences/loops, tempo/meter automation and professional synchronization/control.
Sound	Quick reference notes plus sustained chords/drones, high-quality/original or licensed sounds, temperaments, exercises, scales/intervals/harmonics and auto-follow where acoustically safe.
Analyze	Pitch/waveform, spectrum/harmonics, note history/staff, interval work, onset/body/release, vibrato, level and meaningful comparisons.
Record	Durable audio capture/playback, names/folders, trim, import/export, selected non-destructive transformations and platform-appropriate video later.
Practice	Timers, goals/history, per-note evidence, plans/notes and portable exchange.
Platform	Accessibility, notation/localization, external audio/display/control, offline behavior, data portability and calibrated device quality.

2.2 Where Bocal should surpass

- Interactive, instrument-specific learning rather than a generic pitch display.
- A reverse question: “Given these pressed controls, what note/fingering is this?”
- One workspace per job, with context-specific advanced controls rather than global preference archaeology.
- Equipment/reed/setup context connected to observations.
- Local-first exchange and clear export previews.
- Tone language that supports artistic intent and teacher judgment rather than optimizing one universal brightness target.
- Accessible non-3D equivalents for every model action.

2.3 Research boundary

This is desk research plus build evidence. It is not independent proof that Bocal is more accurate, faster or easier than TonalEnergy. Those claims require moderated task tests and a shared labeled audio/device benchmark. Public documentation also cannot reveal the incumbent's algorithms, licensed recordings, defects, retention or roadmap.

3. Personas and user journeys

Ten primary personas cover the first consumer system. The full table contains exactly five workflows for each in research/Personas_and_50_Workflows.md.

Persona	Five priority jobs
First-week beginner	Find a fingering; confirm intended note; understand a jump; finish a ten-minute plan; retain the lesson cue.

Persona	Five priority jobs
School-band student	Prepare a scale; fix a rehearsal note; practice with pulse; submit minimal evidence; restart after a missed day.
Adult beginner/returner	Rebuild long tones; learn unfamiliar fingerings; maintain a flexible routine; compare setup; prepare for rehearsal.
Advanced/conservatory student	Map intonation; train chord-role tuning; audit attacks/releases; program complex click; study validated extended fingerings.
Professional/doubler	Restore call setup; switch instruments safely; diagnose entrances; run show click; work fully offline.
Private teacher	Demonstrate fingering; capture baseline; assign focused plan; annotate student bundle; switch students/instruments safely.
Ensemble director	Tune/project; teach chord balance; run rhythm rehearsal; distribute assignment; review minimal aggregate evidence.
Section leader/peer coach	Resolve transposition; match articulation; tune a chord; share a pulse; preserve rehearsal decisions.
Parent/guardian	Help start; see effort without surveillance; control minor data; share intended evidence; support maintenance.
Accessibility-first user	Tune without color; use non-3D control list; feel/see pulse; reduce motor/cognitive load; read chart summaries.

3.1 Shared six-stage journey

> Flow: Set context → Model the target → Perform → Interpret evidence → Take one action → Keep or share intentionally

1. **Set context:** player/profile, instrument, transposition, equipment, task and environment.
2. **Model the target:** note/fingering, tone/drone, pulse, excerpt or teacher instruction.
3. **Perform:** microphone/recording state is explicit; the primary correction remains legible.
4. **Interpret:** signed cents, stability, timing and signal shape are explained, not mystified.
5. **Act:** retry, change speed, change target/fingering, annotate or plan the next block.
6. **Retain/share:** save locally; preview the exact bundle before anything leaves the device.

3.2 Critical journey rules

- Switching instrument changes transposition, range/sensitivity, model, personal tendencies and saved workspace together.
- A beginner sees larger tolerances and less density, while a professional can reveal exact cent targets, damping and routing.
- Every analytic card ends with a suggested comparison or retry; no dead-end dashboard.
- A teacher can author interpretation. The app does not overrule contextual pedagogy.
- A minor never needs a public profile, social rank or remote surveillance to complete the learning job.
- 3D tapping is optional. Note and control lists expose the same state to TalkBack, keyboard and switch access.

4. Information architecture and UX specification

Tune

Default surface: large written note, small concert note, frequency, signed cents, explicit flat/centered/sharp word, cent gauge and one Listen button. Instrument, A reference and tolerance are nearby, not buried. Advanced sheet: temperament/key, damping/mode/range, reference routing, display choice and session metrics.

Key UX guardrails:

- Never show an E-flat sax learner only the concert note by default.
- Never use red versus green as the sole result. Use signed value, left/center/right position, text, marker shape and optional haptic.
- Suppress low-confidence flicker; show “play a steady note” rather than rapidly guessing.
- Separate “instant estimate” from “stable note” so range/history is not polluted by transients.

Lab

Default surface: rotatable instrument, note browser, selected control name/finger/side, pressed-control summary and a plain validation badge. Four modes build progressively:

1. **Explore parts** — tap body, neck/headjoint, bell, reed/mouthpiece/bocal, keys/holes.
2. **Note → fingering** — choose written note; model highlights controls and provides text/list equivalent.
3. **Fingering → note** — tap physical controls or checklist; exact/alternative matches appear.
4. **Practice transition** — alternate two validated notes at a chosen tempo; show changed versus held fingers.

The model must never imply manufacturer-exact linkage or repair instructions. Part location can be approximate; educational relationships cannot.

Pulse

Default: tempo, tap, start/stop, meter, beat indicator. Advanced drawer: subdivision/accent/sound/haptic, count-in, random beat/bar silence, preset and sequence editing, tempo/meter automation, drones/cues and integrations. “Practice this” templates translate complexity into jobs such as “three bars on, one bar silent” or “ramp 80→112 over eight repeats.”

Sound

Default: chromatic note/keyboard, octave, sustain and stop-all. Advanced: waveform/sampled sound, chord voices, temperament/key, scale/interval/harmonic exercise, auto-follow and route controls. Bocal needs newly recorded or properly licensed multisamples; competitor sound assets cannot be copied.

Analyze

Default: a synchronized pitch trace and waveform plus a neutral explanation. Advanced: full spectrum/harmonics, onset/body/release, vibrato rate/extent, volume, staff/note transitions, interval trainer, take A/B and equipment-context comparison. “Tone brightness” is a dimension, not a reward axis.

Practice

Default: timer, three focus blocks, note and recent sessions. Advanced: goals, per-note maps, equipment/reed context, recordings, repertoire/wishlist, teacher bundles and longitudinal trends. Streaks, if shipped, need grace/restart design; the primary outcome is returning to practice, not protecting a number.

5. Delivered application capability

5.1 Standalone web build

Location: web-standalone/. Ready build: web-standalone/dist/.

Implemented:

- Browser microphone capture with echo/noise/AGC requests disabled where supported.
- Original YIN-style difference and cumulative-mean-normalized difference detector.
- Reference A, written/concert pitch through the instrument catalog, tolerance, signed cents, confidence, range and recent trace.
- Local Three.js/glTF viewer with orbit/zoom/reset, selectable named controls, 35-instrument selector, key highlighting and alto core note browser/reverse exact match.
- Metronome 30–260 BPM, tap, common meters, 1–4 subdivisions, accent, beat lights, random non-downbeat silence and optional vibration.
- Two-octave oscillator keyboard with sine/triangle/saw/square and equal/just-major/Pythagorean examples.
- Waveform and relative 12-harmonic analysis plus cautious brightness/clarity/vibrato-span descriptors.
- Practice timer/focus plan/history, local lesson note, JSON export and in-tab recording/playback/download.
- Responsive desktop/mobile layout, skip link, semantic controls and device-only data badge.

Known boundaries:

- Browser audio timing and device capture are not a substitute for native low-latency benchmarks.
- The lower detector search bound is around 45 Hz; this is not a C0–C8 parity claim.
- Recordings are ephemeral blobs unless downloaded; durable indexed recording storage is not implemented.
- The JS bundle is about 608 KB minified before gzip and should be code-split after product behavior stabilizes.
- Visual browser QA was limited by the execution browser's WebGL sandbox failure. The build and structural smoke tests pass; physical-browser/device visual QA remains required.

5.2 Native Android source

Location: android/.

Implemented in source:

- Kotlin/Compose six-screen task shell.
- Native AudioRecord input on a worker thread and original YIN-style detector.
- Written/concert display for concert, E-flat and B-flat contexts; A reference and tolerance; pitch gauge and trace.
- Native metronome scheduler with common meter/subdivision, downbeat, visual indicator and vibration.
- Native looping reference-tone engine using static AudioTrack PCM generation.
- Native pitch/confidence analysis snapshot.
- Local practice timer, note, history and JSON text export through the OS chooser.
- Secure local WebViewAssetLoader serving the static lab from app assets; no INTERNET manifest permission.
- Basic unit tests for 440 Hz and silence, build script and APK container validator.

Why this is the source I would build from: audio, haptics, lifecycle, storage and primary navigation are native; the portable 3D renderer is isolated behind a local asset URL and can later be replaced with Filament without changing the catalog/key metadata contract.

What remains before calling it an APK candidate:

1. Clean Android Studio/Gradle sync against SDK Platform 36.
2. Compile and unit-test; resolve any API/version/toolchain integration issue discovered by the real compiler.
3. Install on physical API 26, mid-range current, and high-end current devices.
4. Run mic denial/regrant, route change, long metronome, suspend/resume, WebView renderer death and accessibility checks.
5. Produce a debug APK, run verify-apk.sh, then sign a release/AAB through a private keystore workflow.

5.3 Hosted Sites source

Location: web-source/. It preserves the previous richer Sites/Next/Vinext source and the client-only Three.js load correction. It is useful if Sites hosting is resumed, but the standalone build is the safer downloadable deployment artifact because it has no Worker rendering path or database requirement.

6. 3D asset system

6.1 Inventory

Family	Models	Count
Saxophones	Soprillo, sopranino, soprano, alto, C melody, tenor, baritone, bass, contrabass, subcontrabass	10
Flutes	Piccolo, concert, alto, bass	4
Clarinets	E-flat, B-flat, A, basset horn, alto, bass, contralto, contrabass	8
Double reeds	Oboe, oboe d'amore, English horn, bass oboe/heckelphone, bassoon, contrabassoon	6
Recorders	Sopranino, soprano, alto, tenor, bass, great bass, contrabass	7
Total		**35**

Yamaha identifies six saxophones in widespread use: sopranino, soprano, alto, tenor, baritone and bass. The pack includes those and four useful rare/historic extensions. The broader woodwind taxonomy follows Yamaha's common-family overview; it is not a claim to include every world, historical or experimental woodwind.

6.2 Asset contract

models/catalog.json records:

- stable id, display name and family;
- generator template, stylistic scale and shape;
- instrument key and transpose, defined as sounding/concert semitones relative to written pitch;
- prevalence tier, file path and interactive-control count;
- modelStatus, reviewStatus and optional special mechanism flags.

Every interactive glTF node has a key__ name plus extras such as:

```
{
 "interactive": true,
 "keyId": "lh1",
 "label": "B pearl",
 "finger": "Left index",
 "side": "front",
 "partType": "touch"
}
```

The pack's validate_models.py parses each binary GLB and verifies glTF 2 structure, catalog/file agreement, instrument metadata, declared control counts, key ID presence/uniqueness and node naming. Current result: 35 files and 465 controls validated.

6.3 Educational accuracy release workflow

```
Flow: Author model + map → Structural validation → Family specialist review → Learner task test → Versioned publish → Errata + regression
```

Required evidence per instrument/system:

- Source register, written/sounding convention and exact supported range.
- Primary fingerings plus named alternatives/trills/extended techniques with context.
- Key/control map reviewed on at least two representative real instruments or documented systems.
- One qualified player/teacher authors or reviews; a second independently checks the chart. For school-facing content, include an educator who teaches that level.
- Front/back/left/right silhouette/part review and a “would a learner find the real control?” task.
- Automated transposition and note-map tests; screenshot/interaction regression.
- Visible version/source/reviewer metadata and an erratum path.

Until that gate passes, the UI should say “part exploration; note map awaiting specialist validation.” This is why the other 34 models are not falsely presented as complete fingering tutors.

7. Technical architecture

7.1 Static/local topology

```
Flow: Compose task UI → Native audio + haptics → Local structured data + files → Local 3D lab boundary → WebViewAssetLoader → HTML/JS/CSS + catalog + GLB → OS export/share sheet
```

No server is required for tuner, metronome, tones, models, practice history or file exchange. Optional future integrations must be separate modules with explicit permissions and privacy impact.

7.2 Production module direction

Do not begin with the baseline document's full module count merely because it looks architectural. Extract only tested contracts:

- core-audio: Oboe/AAudio stream, ring buffer, route/capability/latency diagnostics.
- core-pitch: PitchDetector, clean-room YIN/MPM implementations, smoothing/segmentation and fixtures.
- core-analysis: FFT, harmonic/tristimulus/centroid/flux, onset/vibrato and neutral descriptors.
- core-pulse: timeline/preset compiler and native audio/haptic renderers.
- instrument-content: catalog, fingering/part schemas, validation and versions.
- core-data: Room entities/DAOs, app-private audio and migrations.

- bundle: versioned import/export and annotation anchors.
- feature-*: six task surfaces composed from those cores.

The delivered single-module reference keeps these package boundaries without imposing multi-module build overhead before contracts stabilize.

7.3 Audio path

Reference path:

1. Capture 48 kHz mono PCM with AudioRecord/Web Audio.
2. Convert to floating samples.
3. Run a clean-room YIN-style difference function and cumulative mean normalized difference.
4. Select/refine a period, return frequency/confidence.
5. Convert against adjustable A4 to nearest concert MIDI, cents and written pitch using catalog transposition.
6. Publish on the main/UI thread and retain bounded history.

Production path:

- Prefer Oboe/AAudio with native rate, low-latency mode where available, callback-safe ring buffer and device capability diagnostics.
- Keep analysis off the real-time callback; no allocation, locks, UI or file I/O there.
- Test YIN and MPM on the same corpus. SwiftF0/ONNX is an optional detector only after it beats DSP on defined noisy/transition cases without unacceptable onset latency or range loss.
- Use confidence/hysteresis/note segmentation to prevent flicker and keep silence/transients out of statistics.
- Store raw evidence/summary versions so algorithm changes do not silently rewrite history.

Latency must be measured in layers. A 2048-frame window at 48 kHz spans 42.7 ms of signal; frequent overlapping hops can update smoothly but do not make the observation window disappear. Report callback/buffer, algorithmic window/hop, time-to-stable-pitch and UI response separately by device/route.

7.4 Metronome engine

The current engines schedule against audio/system clocks and are sufficient for functional reference. Production parity needs a compiled event timeline:

```
Preset → sections → bars → beats → subdivisions/cues/tones
    tempo function + meter + loop/repeat + silence policy
```

Compile ahead into timestamps; render clicks through a low-latency audio engine; drive visuals/haptics from the same timeline with compensated presentation times. Persist schema/version, not UI state. Add property tests for loops, meter changes and ramps and a 30-minute drift benchmark.

7.5 Tone and recording

- Basic oscillator tones are original and lightweight, but professional comparison needs newly recorded/licensed multisamples, tuned/looped/normalized and documented by instrument/range.
- Prevent reference audio from corrupting microphone analysis: recommend headphones, route/reference cancellation where feasible, and explicit “reference may be heard by tuner” warnings.
- Store recordings app-private by default. Use scoped-storage/file-picker APIs for import/export.
- Use non-destructive edit graphs for trim, pitch/tempo/time changes; retain original and provenance.

- Media3 Transformer is a candidate for platform-supported transformations; specialized musical time/pitch quality needs comparative listening tests and licensing review.

7.6 Data model

Minimum versioned entities:

- PlayerProfile (optional local alias/accessibility/preferences)
- InstrumentDefinition and InstrumentInstance
- EquipmentComponent/Setup and service/reed events
- PracticePlan/Block and Session
- PitchPassage/NoteObservation and AnalysisVersion
- Recording/Take/EditGraph
- MetronomePreset/Group/Timeline
- LessonNote/RepertoireItem
- ContentVersion/Fingering/Control/Review
- Bundle/Annotation/ImportProvenance

Room/SQLite is the production store. Audio/video remain files referenced by database IDs. Export bundles carry `schemaVersion`, app/content/analysis versions, units, transposition convention, timestamps/timezone and an explicit attachment manifest.

7.7 Privacy and security

- Android core has no Internet permission. Audit dependency manifests in every release.
- Microphone use is foreground, visible and user-initiated. Recording is separately explicit and default-off.
- Explain Android backup/transfer behavior; provide inspect, export and delete inside the app.
- Preview exports, especially for minors. Audio is opt-in per bundle.
- No analytics/crash SDK is “free”: any future networked telemetry changes the permission/data story and must be modular/consented.
- A hosted static web app receives normal web request metadata at the host even when audio stays local; the privacy text must distinguish hosting from audio upload.

7.8 Accessibility

- Signed text + direction word + position/shape accompany hue.
- Compose/HTML semantics expose note, cents, state, action and context without announcing every unstable frame.
- Minimum target sizes, large text/reflow, high contrast and reduced motion.
- Beat uses number/shape, optional haptic and sound; no unsafe high-frequency flashing.
- 3D has note list, key list and part list equivalents with logical focus order.
- Charts provide textual min/max/mean/tendency and exportable data.
- Test with TalkBack, Switch Access, magnification and disabled musicians; automated checks are only a floor.

8. Parity status and roadmap

The accompanying CSV is the binding detail. At a product level:

State	Meaning	Examples in handoff
Working reference	Code builds/runs in the web artifact or is implemented in native source; still needs production/device validation	Static app; core web tools; 3D lab; GLB validator; native engine source
Partial	A useful slice exists but not incumbent depth/durability	Tone descriptors; pitch activity; temperament examples; recording; accessibility; haptics
Planned parity	Required to claim broad TE parity	Preset automation; sound library; custom temperaments; advanced analysis/recording; integrations/localization
Validation-gated	Code/geometry cannot ship as educational truth yet	Non-alto note maps; alternates/trills/altissimo; corpus-backed tone interpretation

Milestone 1 — Trustworthy alto launch foundation (planning range: 4–8 focused weeks)

- Build/install Android source; resolve compiler/device issues.
- Replace reference capture/scheduling with measured native low-latency path where evidence justifies it.
- Assemble labeled alto corpus across fundamentals, dynamics, vibrato, attacks, noise and representative devices.
- Two sax educators review alto core map, alternatives and visual/control naming.
- Durable Room practice/recording storage, inspect/export/delete and privacy policy.
- Accessibility-equivalent key/note lists and TalkBack cadence.
- Static host deployment with smoke/asset checks; either redeploy fixed Sites source or use an ordinary static host.

Exit gate: no critical content error; documented detector range/accuracy; real APK installed on device; core journey works offline.

Milestone 2 — Core parity and complete sax family (8–16 additional weeks)

- Sensitivity/damping/mode, per-note activity and intonation map.
- Just/key-role targets, temperament presets and custom editor/import/export.
- Metronome presets/groups, count-in, beat/bar silence, sequences/loops and tempo/meter automation.
- Durable recorder with folders/names/trim/import/export and A/B.
- High-quality original/licensed sax/wave tone library and exercise primitives.
- Specialist-validated soprano/tenor/baritone maps first, then sopranino/bass and rare models; low-A and range conventions.
- Workspaces/presets, external display and robust file bundle exchange.

Exit gate: declared sax feature/content coverage; parity tests pass for P0/P1 rows scheduled to M2.

Milestone 3 — Common woodwind learning system (parallel family tracks, 3–6 months)

- Flute/piccolo, B-flat/A/E-flat/bass clarinet, oboe/English horn, bassoon/contrabassoon and recorder-system note maps.
- Family-specific range/mode/corpus, air/reed/vent explanations and teacher-reviewed exercises.
- Note staff, interval trainer, onset/body/release/vibrato analysis.
- Equipment/reed/bocal/headjoint context and setup comparisons.
- Teacher templates and timestamped bundle annotations.

- Localization/notation packs and full accessibility research.

Exit gate: each enabled family independently passes content, DSP and journey gates. Do not enable all models with one generic sax mapping.

Milestone 4 — Professional ecosystem parity

- MIDI/foot control, Ableton Link and selected external protocols.
- Advanced audio routing, presentation mode and device synchronization.
- Media transformations/video where platform/product evidence supports them.
- Optional organization tooling only if local/file workflows fail real educator research; core remains useful without an account.

Resourcing reality

Full parity plus a validated 35-instrument educational system is not a one-turn build. A credible small team has at least Android/audio engineering, product/web/3D engineering, UX/accessibility, QA/device coverage, a technical artist/content pipeline role and paid family specialists. A solo builder should sequence by validated family and professional job; attempting all rows simultaneously guarantees shallow correctness.

9. Quality and release plan

Automated now

```
# 3D catalog and binary glTF structure
python3 models/validate_models.py

# Static app build and smoke checks
cd web-standalone
npm install
npm run build
npm run test
```

Current results in this handoff: web build succeeds; smoke checks pass for 35 instruments and six workspaces; model validator passes 35 GLBs/465 controls.

Required next suites

- DSP synthetic sweeps, harmonics, noise, vibrato, attacks, two-source interference and silence.
- Golden recordings with note/onset/pitch labels by instrument/register/dynamic; track gross pitch accuracy, raw pitch accuracy, octave errors, cents and time-to-stable.
- Android audio route/device/API matrix; latency and power/thermal profiling.
- Metronome timeline property tests and long-drift/audio-focus tests.
- Catalog transposition tests for every instrument and fingering schema round-trips.
- Screenshot/interaction/accessibility regression on phone/tablet and WebView renderer recovery.
- Bundle migration/fuzz/import provenance and privacy export review.
- Moderated tasks against TonalEnergy for beginner, professional and educator cohorts.

Manual model review sheet

For every model: silhouette recognizable at thumbnail; mouthpiece/reed/headjoint/bocal and bell/body orientation; plausible hand-control distribution; no intersecting/floating controls that imply a wrong action; selectable area discoverability; four-view labels; real-instrument transfer task; non-3D equivalent; review status visible.

10. Delta from the supplied baseline document

The full comparison is research/Baseline_Delta.md.

Strong baseline decisions kept

- Native/on-device core, Kotlin/Compose and min SDK 26.
- Oboe/AAudio production direction and clean-room YIN/MPM licensing discipline.
- SwiftF0/ONNX as an optional candidate rather than a GPL dependency.
- Local Room/files/bundles, color-safe redundant encoding, equipment/journal and phased coaching.
- File-based exchange and modular seams.

Major additions

- Officially sourced incumbent inventory and 84-row parity/acceptance matrix.
- Ten personas, 50 workflows and a shared journey/IA.
- Working standalone product and native source.
- 35-model generator/catalog/validator with 465 controls.
- Content validation/governance and honest gating.
- All saxophone and common woodwind taxonomy, with transposition metadata.
- Build/deployment diagnosis and truthful APK status.
- Milestone/release/benchmark plan.

Corrections

- Under-20-ms “glass-to-glass” cannot be asserted while using 2048–4096-sample pitch windows; latency must be defined and measured in layers.
- No Internet permission materially improves trust but does not eliminate mic/recording/privacy/backup/minor disclosures.
- SwiftF0 published results do not prove sax/altissimo/device superiority.
- “All woodwinds” requires an explicit taxonomy and specialist tracks.
- Visual correctness and fingering correctness are separate approvals.
- Backendless has real tradeoffs for synchronization/organization workflows.
- A large Gradle module graph should be earned through stable tested contracts, not created as ceremony.

11. Delivery map

Path	Purpose
web-standalone/dist/	Ready static HTML/CSS/JS/catalog/GLB build. Serve over localhost/static HTTPS.

Path	Purpose
web-standalone/src/	Standalone TypeScript source; package.json, tests and README included.
models/glb/	35 GLB files.
models/catalog.json	Instrument/model/transposition/review integration contract.
models/generator/	Original pure-Python model generator.
models/validate_models.py	Binary glTF/catalog validator.
android/	Native Android Studio source plus embedded static lab assets.
android/APK_BUILD_STATUS.md	Exact APK blocker; no fake binary.
web-source/	Previous hosted Sites source and client-only 3D load fix.
research/TE_Parity_Matrix.csv	84-row feature status/roadmap/acceptance contract.
research/Personas_and_50_Workflows.md	Ten personas × five workflows.
research/Research_Sources_and_Method.md	Source ledger/method/limitations.
research/Baseline_Delta.md	Detailed delta versus supplied brief.
docs/Bocal_Product_Handoff.md	This handoff.

12. Immediate next decisions

1. **Approve the truth standard:** do not market full parity until scheduled matrix rows pass evidence; do not expose unreviewed note maps.
2. **Choose the Android build environment:** Android Studio or a CI machine with SDK 36/Build Tools 36/JDK 17. Produce the first real debug APK and attach compiler/device results to this handoff.
3. **Recruit two alto reviewers:** one active teacher and one advanced/pro player; lock core naming/fingering conventions and errata process.
4. **Run five comparative tasks:** cold tune, transpose, create practice pulse, record/review, learn a fingering—Bocal versus TonalEnergy, beginner and professional cohorts.
5. **Pick a static deployment path:** ordinary static hosting for dist, or explicitly authorize a fixed Sites redeploy and verify the public checkpoint. Do not keep pointing users to an unverified Worker URL.
6. **Fund the P0 content/DSP test lane before adding more visual polish.** Correctness and transfer to the real instrument are the moat.

Conclusion

Bocal can plausibly beat a mature incumbent on comprehension and learning transfer while eventually matching its professional depth. The opportunity is not a prettier clone. It is a local-first practice operating system where the user begins with an intention, sees the real instrument relationship, receives measured evidence and knows the next action.

The handoff already contains working static software, original source, a native implementation path and a broad educational asset system. What it does not contain is equally important: no fake APK, no unsupported full-parity claim and no invented fingering authority. The next successful increment is a compiled/device-tested Android build and specialist-certified alto experience, followed by parity and instrument families through explicit gates.

Appendix A. Bocal personas and 50 priority workflows

This is the complete primary persona set for the woodwind-learning product boundary—not a claim that every human who might open a tuner is identical. Repair technicians, instrument retailers, composers/arrangers, accompanists and researchers are secondary stakeholders; their specialized workflows should not distort the first consumer product until evidence shows a recurring job.

Every persona has exactly five priority workflows. “Success evidence” is what the product should observe or let the person confirm; it is not a gamified score invented for its own sake.

P1 — First-week woodwind beginner

Context: little vocabulary, fragile sound production, unsure how the instrument maps to notation. Needs immediate comprehension and low shame.

#	Trigger and job	Bocal workflow	Success evidence
1	“Which key makes this note?”	Choose instrument → choose written note → 3D model rotates to usable view → required controls illuminate and name the fingers → learner mirrors it.	Learner can reproduce the fingering twice without reopening the hint.
2	“Am I playing the intended note?”	Open Tune → instrument transposition is preselected → play → see large written note, concert note in secondary text, sharp/flat direction and cent distance.	Intended written note is detected and held inside a generous beginner window.
3	“Why did the note jump?”	Tune detects an octave/adjacent-note change → show likely causes as hypotheses: air speed, octave key/vent, leak or wrong finger → link to the exact key map.	Learner identifies whether the jump follows fingering or voicing and retries.
4	“How do I practice for ten minutes?”	Pick Beginner plan → 2-minute setup, 4-minute long tones, 3-minute three-note pattern, 1-minute listen-back → timer advances one task at a time.	A complete short session is saved; no setup screen is needed next time.
5	“What should I remember after the lesson?”	Teacher/learner writes one short note and pins one 3D fingering → next session starts with that cue.	Learner can state the focus and open the pinned note in one tap.

P2 — School-band student, often under 18

Context: assignment-driven, limited practice time, shared rehearsal vocabulary, possible school-managed device and guardian/privacy constraints.

#	Trigger and job	Bocal workflow	Success evidence
1	Prepare assigned scale	Select instrument and scale → hear tonic/drone → see note sequence and 3D fingering on demand → tuner follows each stable note.	Correct order, range and written pitches; trouble notes are identified without a public leaderboard.

#	Trigger and job	Bocal workflow	Success evidence
2	Fix a rehearsal intonation note	Director says “written F-sharp is high” → search written F-sharp → play against target/drone → compare attack and sustained body.	Median cent tendency moves toward the chosen tolerance across three attempts.
3	Practice with a metronome	Open saved assignment pulse → count-in → play with audible/visual/haptic beat → optional random silence tests internal time.	Student returns on time after silent bars and records the comfortable tempo.
4	Submit evidence without an account	Run timed assignment → add a note → export a .bocalbundle/JSON summary through the OS share sheet, with audio only if explicitly included.	File identifies app/schema, instrument, task, duration and metrics; recipient can import locally.
5	Recover after a missed day	Home shows weekly minutes and last focus, not a broken-streak punishment → choose a five-minute restart block.	Student resumes; product measures return-to-practice rather than only consecutive days.

P3 — Self-directed adult beginner or returning amateur

Context: autonomous, may practice irregularly, wants insight without feeling infantilized, often spends on equipment and lessons.

#	Trigger and job	Bocal workflow	Success evidence
1	Rebuild embouchure and air support	Start long-tone routine → reference pitch/drone → live cent and stability trace → compare today with personal baseline.	Sustained duration and variability improve without defining one “correct” timbre.
2	Learn unfamiliar fingerings	Search a note or tap a control on the 3D model → compare primary and validated alternative fingerings → save favorite by context.	Player recalls the fingering in repertoire and knows why an alternate is used.
3	Keep an achievable routine	Compose a 15/30/45-minute plan from blocks → local reminders are optional → timer and journal capture actual time.	Planned versus completed work is visible without requiring cloud sync.
4	Compare reed/mouthpiece setup	Select equipment setup → record the same protocol → compare pitch stability, response notes and player comments; avoid declaring a universal winner.	Player can associate observations with a specific reed/mouthpiece and date.
5	Prepare for community rehearsal	Load set-list notes → tune known problem pitches → run tempo ramps for difficult bars → export a compact rehearsal checklist.	Player reaches target tempo and arrives knowing personal intonation tendencies.

P4 — Advanced, college or conservatory student

Context: needs repeatable measurement, alternate fingerings, excerpts, intonation systems and teacher-facing evidence; rejects toy-like coaching.

#	Trigger and job	Bocal workflow	Success evidence
1	Map instrument intonation	Run chromatic protocol by register/dynamic → per-note cent distribution stored with equipment/context → review heat map.	Enough stable samples exist per note to distinguish tendency from one attempt.
2	Train chord-role intonation	Choose temperament/key/degree → drone/chord target changes cent offset by function → play and record tendency.	Student can repeat root/third/fifth placements and explain the chosen system.
3	Audit attacks and releases	Record excerpt or isolated notes → analysis marks pitch at onset, body, release, amplitude and vibrato span → compare takes.	Specific transition is identified and improves across controlled takes.
4	Build an advanced click track	Create sections with meters, tempo changes, accelerando/ritardando, count-ins and silent bars → loop chosen section.	Preset runs deterministically and can be exported/imported.
5	Study altissimo/extended fingering	Select instrument/model and note → view specialist-validated base/alternate fingering, vent explanation and personal annotation → record result.	Fingering is associated with instrument/setup and works in the target transition, not just isolation.

P5 — Professional performer and woodwind doubler

Context: speed, reliability, several transposing instruments, rehearsal/performance use, external gear, and minimal screen attention.

#	Trigger and job	Bocal workflow	Success evidence
1	Fast pre-call setup	Open pinned “Call” workspace → A reference, instrument/transposition, temperament and mic route restore → one-tap tuner/metronome overlay.	Ready in seconds with no accidental stale instrument setting.
2	Switch instruments safely	Tap next instrument in set order → written/concert display, range sensitivity, fingering model and personal tendencies change together.	No transposition ambiguity; switch state is prominent and spoken to accessibility services.
3	Diagnose a difficult entrance	Record several entrances → inspect note-start pitch/latency and waveform → compare at performance dynamic.	Best take is named, annotated and retrievable at rehearsal.

#	Trigger and job	Bocal workflow	Success evidence
4	Follow a complex show click	Run programmable preset group with meter/tempo changes, count-ins, cues and haptics → external MIDI/foot control when available.	Cues stay aligned for the full sequence and external control is recoverable.
5	Work offline at a venue	All core tools, presets, models and recordings function without sign-in/network → export via cable/share sheet later.	No dependency on venue connectivity; data remains accessible and portable.

P6 — Private teacher or studio instructor

Context: sees varied ages and instruments, has little transition time, needs language that teaches rather than merely scores.

#	Trigger and job	Bocal workflow	Success evidence
1	Demonstrate fingering visually	Select student instrument/note → cast or hand over the rotatable 3D view → isolate controls and name fingers/parts.	Student can point to and operate the correct mechanism on the real instrument.
2	Establish a diagnostic baseline	Run a short teacher protocol: long tone, scale, articulation, excerpt → tag conditions → review signal evidence.	Baseline is repeatable and teacher, not algorithm, writes the interpretation.
3	Assign focused practice	Build a plan from exercises/presets → add success criteria and optional audio example → export locally.	Student imports the exact plan without an account and completes the intended sequence.
4	Review student work	Import bundle → listen/inspect selected moments and trends → attach timestamped comments → return bundle.	Comments anchor to the right take/time/note and preserve schema/version.
5	Manage a multi-instrument lesson day	Use fast profile switching without exposing one student's data to another → reset live session but retain private histories separately.	Correct student/instrument context is visible; accidental cross-student export is prevented.

P7 — Ensemble director, conductor or band/orchestra educator

Context: full-room visibility, fast shared pitch/rhythm references, score-order transpositions, projection and assignment administration.

#	Trigger and job	Bocal workflow	Success evidence
1	Tune an ensemble	Director mode shows large concert pitch/bar, reference A and optional drone on external display → individual instruments keep written labels locally.	Shared concert target is legible across the room; no student's microphone data is collected centrally by default.
2	Teach chord balance	Select chord and temperament → display each scale degree's target offset → sections sustain in sequence/combination.	Players can identify role and adjust toward the chosen tuning model.

#	Trigger and job	Bocal workflow	Success evidence
3	Run rhythm rehearsal	Build or load meter/tempo sequence → voice count-in and visual flash → silence selected bars/parts.	Ensemble maintains pulse through removed cues and re-enters together.
4	Create an assignment	Define instrument-aware scale/excerpt workflow with tempo/tolerance bands → export one template through school-approved channel.	Students import the same task; transposition is resolved per instrument.
5	Review aggregate progress ethically	Students optionally submit minimal summaries → director views completion/problem-note distribution without public ranking or continuous surveillance.	Data minimization and consent are clear; action is a rehearsal plan, not a punitive score.

P8 — Section leader, chamber partner or peer coach

Context: collaborative rehearsal without formal teacher authority; needs shared vocabulary and low-friction comparison.

#	Trigger and job	Bocal workflow	Success evidence
1	Resolve written/concert confusion	Select each instrument → Bocal shows the same sounding pitch with each written equivalent and fingering link.	All players name one concert target and their own written note correctly.
2	Match articulation	Record short reference and responses → align onset/waveform envelopes and listen A/B at matched level.	Group agrees on a repeatable attack/release description and improves ensemble timing.
3	Tune a chord	Choose concert chord/voicing → assign degree targets → each player sees instrument-transposed note.	Beats/cent tendencies reduce according to the explicitly chosen temperament.
4	Share a rehearsal pulse	One player exports a click/preset group → peers import locally or synchronize via future Link support.	Meter/tempo/cues are identical across devices.
5	Capture decisions	Save one shared rehearsal note with measure, fingering/intonation choice and next tempo → export to group channel.	Next rehearsal starts from the recorded decision, not memory alone.

P9 — Parent, guardian or practice supporter

Context: wants to support routine and safety without becoming a surveillance dashboard or needing musical expertise.

#	Trigger and job	Bocal workflow	Success evidence
1	Help start a session	Child opens a teacher-provided plan; supporter sees only duration and next action, not complex analytics.	Setup friction is removed and the learner retains control of playing.

#	Trigger and job	Bocal workflow	Success evidence
2	Understand whether practice happened	Optional device-only weekly summary shows completed minutes/tasks and learner-authored note; no public rank.	Conversation centers on effort and next help, not a streak penalty.
3	Protect a minor's data	Onboarding states mic purpose, recording default-off, local storage, export contents and deletion → guardian/learner choose.	No account or background upload; consent choices can be revisited and data deleted.
4	Prepare for a lesson	Learner selects a take/question to share; supporter uses OS share sheet only after a content preview.	Only intended files/metrics leave the device.
5	Support equipment upkeep	Local reminders explain reed rotation, swabbing and service notes without diagnosing damage.	Maintenance action is completed; repair concerns route to teacher/technician.

P10 — Accessibility-first learner or professional

Context: may use TalkBack, magnification, switch access, reduced motion, high contrast, haptics, one-handed adaptation, captions/text equivalents or hearing protection. Accessibility needs can coexist with any persona.

#	Trigger and job	Bocal workflow	Success evidence
1	Tune without relying on color	Cent direction is position + signed text + shape + optional haptic pattern; TalkBack announces note/state at a controlled cadence.	User identifies flat/centered/sharp without hue and without speech flooding.
2	Use the 3D lab without precise touch	Note list and logical key list provide the same state as direct 3D tapping; focus order names hand/finger/part; reset-view is accessible.	Every 3D educational action has a non-3D, keyboard/switch-accessible equivalent.
3	Follow pulse without hearing clicks	Full-screen visual beat, high-contrast number/shape and capability-aware haptics; no flashing above safe frequency.	User follows downbeat and subdivision through chosen modality.
4	Reduce cognitive/motor load	Beginner/Focus mode shows one primary action, larger targets, persistent instrument context and undo; motion can be reduced.	Core task completes without timeouts, drag-only gestures or hidden long presses.
5	Review analysis accessibly	Charts have text summaries, range/mean/trend descriptions and data export; audio examples have named context.	User reaches the same decision from non-visual or non-audio representation.

Cross-persona journey architecture

All 50 workflows reduce to six repeatable stages:

1. **Set context** — person, instrument, transposition, task, environment and equipment.
2. **Model the target** — note/fingering, reference sound, pulse, phrase, or teacher instruction.

3. **Perform** — microphone/recording is explicit and visible; the UI keeps the main correction readable.
4. **Interpret** — evidence is descriptive (cent direction, stability, waveform, time), while artistic/pedagogical judgment remains human.
5. **Act** — retry, slow down, change fingering, change target, annotate, or schedule the next block.
6. **Retain/share intentionally** — save locally; preview and export only the selected evidence.

This journey is the intended information architecture. Bocal should not expose TonalEnergy's entire capability inventory as a wall of global settings. A user enters through one of six verbs—Tune, Learn, Pulse, Sound, Analyze, Practice—and advanced controls appear inside the task or a saved workspace.

Persona-derived non-negotiables

- Written and concert pitch must never be visually ambiguous.
- Instrument/profile context must persist, be prominent, and switch atomically.
- A “score” cannot replace the underlying signal or teacher/player interpretation.
- Minors and students need local-first defaults, explicit export previews and no dark-pattern streak pressure.
- Every critical color status needs text/position/shape; every 3D action needs a structured-list equivalent.
- Teachers/directors need templates and exchange; they do not automatically need a centralized surveillance backend.
- Professionals need preset recall, deterministic behavior, external control and genuine measured latency.
- Instrument content has a governance workflow: author → source → specialist review → test → version → publish → erratum.

Appendix B. Bocal research sources and method

Research snapshot: 10 August 2026. This is competitive/product research, not a legal opinion or a claim that every platform-specific TonalEnergy feature behaves identically on Android and iOS.

Method

1. Treat TonalEnergy's own Android/Desktop user guide and mobile product page as the source of truth for the incumbent capability set. Store-listing copy was used only as a cross-check.
2. Treat instrument-maker educational references as a starting point for family, part, transposition, and fingering facts. Do not infer a complete fingering curriculum from a picture or from geometry.
3. Separate observed competitor facts from Bocal recommendations. A feature in a public guide is a parity requirement; its priority is still a product judgment.
4. Mark platform qualifiers. TonalEnergy itself labels some items as iOS-only or platform-dependent.
5. Do not copy competitor code, sounds, visual assets, text, or interaction trade dress. The Bocal implementations and 3D models are original reference work.

Primary competitor sources

- TonalEnergy Android/Desktop user guide — detailed table of contents and operating descriptions for tuning, temperament, pitch tracking, activity, metronome, preset sequencing, tone generation, analysis, recording, custom views, preferences, remote control, and MIDI.
- TonalEnergy mobile product page — summarizes the C0–C8 Target Tuner, provided/custom temperaments, mode/range, reference A and transposition; tone sound library and exercise surfaces; waveform/spectral/staff/interval analysis; programmable metronome and silence/count-in functions; recording/editing; integrations; preferences; accessibility.
- TonalEnergy home and education navigation — corroborates retail, desktop, education, organization/student, parent, and support contexts.
- TonalEnergy information for parents — confirms the parent/legal-guardian and under-13 privacy/consent context. Bocal's no-account local-first design can reduce, but does not erase, child-privacy responsibilities.

Capability interpretation

The official guide's breadth is the reason this handoff does not call a basic tuner “parity.” The documented product includes:

- Target and Bar tuner styles; cent resolution to tenths; volume and tone meters; note-start and last-interval options; transposition, notation, adjustable reference A, modes, ranges, damping, equal/just/provided/custom temperaments, pitch tracking, timed events, per-note activity, calendars and goals.
- Metronome tempo entry/tap, beat chooser, meters, subdivisions, accents, multiple sounds, visual flash, count-in, animated display, programmable presets, sequences, loops and groups, tempo changes, silence exercises, Ableton Link and BodyBeat.
- Tone generation through wheel, keyboard and pitch grid; synthesized and sampled sound choices; auto-reference and exercise creation.
- Waveform/pitch, spectral, harmonic-energy, note-staff, interval-training and mixer surfaces.
- Audio recording/playback, folders, trim/rename/delete, import/export, tempo and pitch adjustment, and platform-dependent video.

- MIDI/remote control, external displays and audio devices, languages/notation systems, VoiceOver and sharing.

These statements describe TonalEnergy's documented scope, not an independent quality benchmark. The parity matrix records implementation status feature by feature.

Instrument and pedagogy sources

- Yamaha saxophone family — states that Adolphe Sax originally conceived 14 family members and identifies six in widespread use, high to low: sopranino, soprano, alto, tenor, baritone and bass.
- Yamaha saxophone fingering — provides a basic chart, alternatives where available, and states that basic finger work is shared across saxophones, with the baritone low-A exception.
- Yamaha saxophone transposition — explains written versus sounding pitch, the shared fingering rationale, and B-flat/E-flat examples.
- Yamaha fingering hub — routes to flute, recorder, clarinet, saxophone, oboe and bassoon fingering references.
- Yamaha woodwind-family overview — identifies the common flute/piccolo/recorder/clarinet/saxophone/oboe/bassoon families and explains tone holes, keyed and open-hole mechanisms, single and double reeds, and common variants.

The model catalog intentionally goes wider than only the most common instruments. “Covered” means a selectable educational model exists, not that a complete note chart has been certified. A qualified teacher/player must approve every family-specific map, alternate fingering, trill, range, octave convention, and manufacturer mechanism before publication.

Android and web platform sources

- Android low-latency audio with Oboe and AAudio guide — production capture/rendering direction and device-variability caveats.
- Android AudioRecord — platform input API used by the compact native reference.
- Android haptic effects — capability-aware waveform/composition guidance.
- WebViewAssetLoader and local WebView content — secure local asset hosting used for the Three.js laboratory without an Internet permission.
- Android offline-first guidance and Room — target local data architecture.
- Media3 Transformer — future on-device trim/transcode/playback transformation path.
- Compose accessibility and adaptive layouts — semantics, target size, state descriptions, and phone/tablet layout direction.
- AGP 9.3 release notes — as of the research date: Gradle 9.5.0, Build Tools 36.0.0, JDK 17, maximum API 37.
- Built-in Kotlin migration — AGP 9+ enables Kotlin compilation without the legacy Android Kotlin plugin.
- Kotlin releases — 2.3.21 is the latest stable entry in the source as of the snapshot.
- AndroidX WebKit releases — 1.16.0 stable; min SDK 24.
- AndroidX Lifecycle releases — 2.11.0 stable at the snapshot.
- Compose BOM mapping — source for the pinned Compose UI/foundation 1.11.4 and Material 3 1.4.0 reference versions.
- glTF — royalty-free runtime 3D delivery format used by the model pack.
- Filament — production alternative for native physically based glTF rendering when replacing the local WebView is worth the cost.

DSP and accessibility sources

- Alain de Cheveigné and Hideki Kawahara, YIN, a fundamental frequency estimator — algorithmic basis for the clean-room difference/CMND detector.
- Philip McLeod and Geoff Wyvill, A Smarter Way to Find Pitch — MPM option for future comparative tests.
- Sebastian Nieradzik, SwiftF0 — small neural pitch model candidate. Published benchmarks are not a substitute for sax/woodwind device testing.
- ONNX Runtime mobile — possible deployment layer for a validated neural model.
- W3C, Use of Color and Non-text Contrast — status must not depend on hue alone; controls/indicators need sufficient contrast.

Research limitations and next evidence

- No independent comparative latency/accuracy benchmark was run against TonalEnergy. The next defensible claim requires the same labeled woodwind corpus and device set for both apps.
- Public feature documentation cannot reveal competitor algorithms, sound licenses, retention, defect rates or internal roadmap.
- Fingering charts vary by system, range, register and pedagogical convention. An authoritative-looking interactive model makes wrong content more harmful, not less.
- The 3D pack received structural validation only. It needs visual review from front/back/left/right plus a pedagogical review checklist per family.
- A genuine APK build and device run were blocked by the absent Android SDK/build toolchain in this environment; the source and deterministic build/verification scripts are supplied.

Companion parity matrix at a glance

The companion CSV contains 84 capability rows. It is the testable scope ledger; this summary shows row count by domain.

Domain	Capability rows
Tuner	22
Metronome	15
Tone	8
Analysis	7
Recording	6
Practice	6
3D learning	9
Platform	11